Implementation Plan — Halal Business Sponsored Website

Implementation Plan — Halal Business Sponsored Website

Cloudflare-hosted build plan, derived from `Brief_HalalBusinessSponsoredWebsite_Numan.docx` and `Halal Business Design System.html`.

1. What the source docs establish

Brief — Next.js, no auth, no payments. Two pages (landing + Programme Terms) plus an application form. 16 landing sections, 14 Terms items, a 20-field application data model, 8 non-negotiable rules, and a 3-milestone build order with an explicit QA checklist. Open dependency: sub-brand name/logo/domain (`[Halal Brand]`) not yet decided — blocks Milestone 3 only.

Design system — CSS-custom-property token system already fully specified: emerald palette (`--color-primary: #10B981` family), Georgia/EB Garamond display + Merriweather body, spacing/shadow/radius scale, and working Button/Input/Card/Badge components. This plan treats it as the source of truth for styling — port the tokens verbatim rather than re-deriving them or introducing Tailwind's own scale.

2. Stack decisions (Cloudflare-specific)

Concern	Choice	Why
Framework hosting	Next.js App Router on **Cloudflare Workers via `@opennextjs/cloudflare`**	Cloudflare's current-recommended path for full Next.js (Server Actions/Route Handlers, ISR, middleware). The older `@cloudflare/next-on-pages` is in maintenance mode and drops features this app needs (API routes with Node-ish deps for email).
Database	**Cloudflare D1** (SQLite), accessed via **Drizzle ORM**	Satisfies the brief's "stored durably" requirement natively in-stack; free tier covers this volume (≤5 accepted/month) easily; schema migrations via `drizzle-kit`.
Team review workflow	D1 as source of truth + an **admin list page at `/admin/applications`, gated by a signed session-cookie login at `/admin/login`**	Brief explicitly leaves "spreadsheet vs. review screen" open; this plan selects the review screen (§10 item 5) — avoids a Google service-account credential to manage and lets `status` / `internal_score` / `internal_notes` / `reviewer` be edited in place.
Bot/spam protection	**Cloudflare Turnstile** (managed widget) + honeypot field	Native to Cloudflare, free, no CAPTCHA UX tax. Satisfies "basic spam protection."
Rate limiting	Cloudflare **Rate Limiting rule** on `/api/apply`, backed by a KV counter as fallback	Belt-and-suspenders against scripted submission spam.
Transactional email	**Hostinger mailbox (Titan Mail) via SMTP**, sent from the Worker using Cloudflare's TCP Sockets API (`cloudflare:sockets`) through a thin SMTP client such as `worker-mailer`	Team already owns a Hostinger-hosted mailbox on the project domain — no separate email-service account/API key to provision. Hostinger has no HTTP send API (unlike Resend), so the Worker opens an authenticated STARTTLS (port 587) or SMTPS (port 465) connection directly. Two branded HTML templates (team notification + applicant confirmation) — see §5a.
Analytics	**Cloudflare Web Analytics**	Cookie-free, so the brief's "cookie consent if used" clause becomes moot — avoids building a consent banner. Swap for GA4 only if the team specifically wants it (then a consent banner is mandatory).
Admin auth	**Custom login page (`/admin/login`) + signed session cookie**, checked against `ADMIN_USERNAME` / `ADMIN_PASSWORD` Worker secrets in `proxy.ts` middleware	Originally planned as Cloudflare Access (Zero Trust), but that needs a Zero Trust org already provisioned on the account. First built as HTTP Basic Auth (the browser's native prompt) as a quick code-only equivalent, then replaced with a designed login page for a better, on-brand experience — still fails closed if secrets are unset, still no dashboard dependency, still matches "no auth system… on this site" for the *public* site. Session cookie is a stateless HMAC-signed token (`lib/admin/auth.ts`), not a sessions table — reasonable for one shared admin login with no per-user accounts. Revisit if Zero Trust gets set up later.

Concern	Choice	Why
DNS / domain	Cloudflare DNS, custom domain bound to the Worker once `[Halal Brand]` domain is finalized	Blocks Milestone 3 only, per brief.
Secrets	`wrangler secret put` per environment; `.dev.vars` for local dev, gitignored	Matches the brief's "no API keys or secrets committed to GitHub" rule.

3. Repo structure

```
app/
  (marketing)/
    page.tsx                 # landing page — 16 sections
    programme-terms/page.tsx # 14 Terms items
    apply/page.tsx           # application form
  api/
    apply/route.ts           # POST handler: validate → Turnstile verify → D1 insert → emails
  admin/
    applications/page.tsx    # login-gated review screen (proxy.ts + /admin/login)
    applications/[id]/page.tsx
  sitemap.ts
  robots.ts
components/
  ui/                        # Button, Input, Textarea, Card, Badge, FormField — ported from design sy
  sections/                  # one component per landing section
lib/
  db/                        # drizzle schema + client (D1 binding)
  email/
    client.ts                # SMTP client (worker-mailer over cloudflare:sockets), Hostinger mailbox
    templates/                # branded HTML templates — see docs/email-templates/ for source mockups
  validation/                # zod schemas (client + server share these)
  turnstile.ts               # server-side siteverify call
  config.ts                  # single source for capacity number + [Halal Brand] name/domain
styles/
  tokens.css                 # ported 1:1 from the design system's CSS variables
  globals.css
drizzle/                     # generated migrations
wrangler.jsonc               # D1 binding, KV binding, env vars
```

4. Data model (D1 / Drizzle)

Maps directly to the brief's Data Model table:

```sql
CREATE TABLE applications (
  id TEXT PRIMARY KEY,                 -- uuid
  business_name TEXT NOT NULL,
  business_type TEXT NOT NULL,         -- enum: 7 niches + 'other'
  years_operating TEXT NOT NULL,
  current_website_url TEXT,            -- nullable
  instagram_handle TEXT,
  facebook_handle TEXT,
  activity_level TEXT,
  google_profile_url TEXT,
  google_review_count INTEGER,
  services_description TEXT NOT NULL,
  biggest_challenge TEXT NOT NULL,
  content_readiness TEXT NOT NULL,     -- yes / partially / no
  credentials TEXT,                    -- halal cert (food) or generic (others), per Decision 6
  contact_name TEXT NOT NULL,
  contact_email TEXT NOT NULL,
  contact_phone TEXT NOT NULL,
  consent_terms INTEGER NOT NULL,      -- boolean, required
  consent_feedback INTEGER NOT NULL,   -- boolean, required
  status TEXT NOT NULL DEFAULT 'new',  -- new/in_review/accepted/waitlisted/declined/completed
  internal_score INTEGER,
  internal_notes TEXT,
  reviewer TEXT,
  created_at TEXT NOT NULL DEFAULT (datetime('now'))
);
```

`consent_terms` and `consent_feedback` are enforced required server-side (zod `.refine`) regardless of client state — never trust the checkbox alone.

5. Application form flow

1. Client-side: Zod schema validates on submit (fast feedback), Turnstile widget completes invisibly/interactively.
2. `POST /api/apply`: re-validate every field server-side (same Zod schema, shared module) → verify Turnstile token against Cloudflare's siteverify endpoint → reject if honeypot field is non-empty (silently 200, to not tip off bots) → insert into D1.
3. Fire two branded emails via the Hostinger mailbox (SMTP) in parallel: team notification (all fields + admin link) and applicant confirmation (warm, sets expectation on next steps — no promises the brief doesn't make).
4. Return success → client shows confirmation state (not just a toast — brief's QA point 7 wants a "confirmation shown to the applicant").

Acceptance test for Milestone 2 (per brief): a test submission must appear where the team reviews within a minute, with validation and confirmation both working — this flow satisfies that with D1 write (near-instant) + SMTP send (typically seconds, validated by the Milestone-2 spike in §10 item 8).

5a. Branded email templates

Two required emails, both built as self-contained HTML (inline styles, table-based layout, Georgia/serif web-safe fallback — matches the design system's `--font-body` stack, since email clients won't reliably load Google Fonts) so they render consistently across Gmail, Outlook, and Apple Mail. Emerald palette (`--color-primary #10B981` / `--color-primary-dark #059669` / `--color-primary-pale #ECFDF5`) and neutral text scale ported straight from the design system tokens.

Source mockups (with `{{placeholder}}` merge fields) live in `docs/email-templates/` : - `applicant-confirmation.html` — sent to the applicant on submit. Warm, brief, sets expectation without over-promising; includes the programme disclosure line per Non-Negotiable Rules. - `team-notification.html` — sent to the internal review mailbox. Every application field laid out for fast scanning, plus a direct link into `/admin/applications/[id]` .

Both share a common visual shell (header wordmark, emerald accent bar, footer with Takween attribution) so a future third template (e.g. an acceptance/waitlist/decline status update) can reuse the same shell without new design work. At build time these become either plain string-interpolated HTML sent via the SMTP client, or React-Email components rendered to a string before sending — either is compatible with the SMTP transport; pick based on whichever the team finds easier to maintain.

6. Page-by-page content plan

Landing page and Programme Terms page are built as data-driven section components under `components/sections/` , each taking its copy from a single content module (not hardcoded across the tree) so the capacity number, `[Halal Brand]` name, and disclosure lines only need updating in one place (`lib/config.ts`) — directly satisfying the brief's Decision 1 and Decision 3 requirements and QA point 5.

Landing sections 1–16 and Terms items 1–14 map one-to-one to the brief's tables — no new decisions needed there, just faithful implementation. The three disclosure placements (footer, Terms page top, near the apply-form submit button) are rendered from one shared `<Disclosure>` component/string so wording can't drift between locations.

7. Security & the brief's Non-Negotiable Rules

- Capacity figure: single value in `lib/config.ts` , read everywhere it's displayed — never a second hardcoded copy.
- No fabricated testimonials: portfolio/testimonial section is conditionally rendered — renders nothing (not placeholder content) until real, consented entries exist in D1/CMS.
- Disclosure visible pre-submission: rendered directly above the Turnstile/submit control on `/apply` , not just linked.
- Consent capture: `consent_terms` + `consent_feedback` both required booleans, stored per-record, tied to the exact wording from the brief (§14) — the review-permission checkbox (Decision 5's Google-review ask) stored as a separate optional field, never merged into the required consent checkbox.
- Secrets: Turnstile secret key, Hostinger SMTP credentials, admin login credentials, D1/KV bindings all via `wrangler secret` / dashboard env vars — nothing in the repo. `.dev.vars` is gitignored.

8. Analytics & SEO

- Cloudflare Web Analytics beacon in root layout (no cookie banner needed).
- `app/sitemap.ts` and `app/robots.ts` route handlers (Next.js built-ins) covering the two public pages; `/apply` and `/admin/*` excluded/noindexed.
- Per-page `metadata` export (title, description, OG image) via Next.js Metadata API.

9. Accessibility & performance

- Keyboard-navigable forms with visible focus states (already in the design system's `:focus` box-shadow token).
- Label-for associations on every form field; error messages tied via `aria-describedby`.
- Color contrast: emerald-on-white and the neutral text scale from the design system meet WCAG AA — verify computed values in Milestone 1 QA, not just assume.
- Mobile-first responsive check (brief's Milestone 1 acceptance criterion) via Chrome DevTools device emulation + a real device pass before submission.
- Cloudflare Workers' edge execution + D1 co-location keeps API latency low without extra caching work; static landing/Terms pages should still be prerendered (SSG) since content changes rarely.

10. Decisions

Pending — needs Daniyal's sign-off (from the brief)

These are content/business calls, not this plan's to make — recommendation adopted as the working assumption until confirmed: 1. Disclosure wording/placement (brief's Decision 1) — recommendation adopted as-is above. 2. Hero headline (Decision 4) — adopted. 3. Halal/credentials field behavior (Decision 6) — adopted. 4. Google review ask as separate optional question (Decision 5) — adopted.

Resolved — Cloudflare-specific

Technical calls this plan makes directly, so build isn't blocked waiting on them:

5. **Admin review workflow → selected: `/admin/applications` screen over D1, gated by a custom login page + session cookie (originally planned as Cloudflare Access, then HTTP Basic Auth, before landing on a designed login page — see §2's Admin auth row).** Rejected the Google Sheets sync alternative — it would add a service-account credential to manage (a secrets-hygiene risk in itself) for no gain, since D1 already needs to be the source of truth and the screen supports inline `status` / `internal_score` / `internal_notes` / `reviewer` editing the brief calls for. Only revisit if the team finds the screen actively worse than their spreadsheet after using it in Milestone 2.
6. **Hostinger mailbox → selected: send from a dedicated address on the project's Hostinger-hosted domain** (e.g. `apply@[Halal Brand domain]`, not a personal inbox — keeps replies and deliverability reputation scoped to the programme). What's left is provisioning, not a decision: SMTP credentials (or app-specific password) via `wrangler secret put`, and confirming SPF/DKIM/DMARC are correctly set in Hostinger's DNS — tracked as a pre-Milestone-2 checklist item.
7. **Cloudflare account/zone access → resolved.** Logged in via `wrangler` as `takweencentreuk@gmail.com` (Account ID `8696616de833631acaef3e2d03464394`), token scoped for D1, Workers, KV, Pages, zone (read), SSL certs, email routing/sending, and challenge widgets

(Turnstile) — covers every binding/service this plan needs. No further access to arrange.

8. **SMTP-over-Workers → selected: `worker-mailer` over `cloudflare:sockets` as the default transport**, not a fork in the road. Still run a short validation spike early in Milestone 2 (confirm it authenticates to Hostinger and delivers both templates reliably) before depending on it in production, but that's a build task, not a blocking decision. The serverless-relay fallback stays documented in case the spike fails — out of scope otherwise.

11. Milestones (brief's build order, Cloudflare deliverables added)

Milestone 1 — Landing + Terms Build: all 16 landing sections + 14 Terms items, ported design-system tokens/components, deployed to a Cloudflare Workers staging URL (`*.workers.dev` or a Cloudflare Pages preview) via GitHub-connected Workers Builds — satisfies the brief's "staging link for review." Acceptance: content matches brief section-by-section; disclosure visible on both pages; mobile-friendliness check passes.

Milestone 2 — Application system Build: D1 schema + migrations, `/api/apply`, Turnstile + honeypot + rate limiting, branded Hostinger SMTP emails (§5a), admin review screen (§10 item 5). Acceptance: test submission appears within a minute where the team reviews; required-field/consent validation works; applicant sees clear confirmation.

Milestone 3 — Launch readiness Build: custom domain bound in Cloudflare DNS (blocked on `[Halal Brand]` decision), Cloudflare Web Analytics live, sitemap/robots/metadata in place, full Non-Negotiable Rules + QA checklist pass. Acceptance: live on the agreed domain; every brief rule checked off.

11a. 10-hour build sprint

This is a compressed vertical slice, not the full 3-milestone scope above delivered faster. 10 hours is enough for one person (or one person pairing with AI) to get a real end-to-end flow working — landing + Terms content, a validated application form, D1 storage, both branded emails, and a working admin screen, deployed to a staging URL — but not enough for the polish, hardening, and sign-off steps the full plan calls for. Treat this as "prove the whole system works," with the deferred list below picked up afterward.

Assumes what's already done: design tokens + both email templates built (§5a), Cloudflare login confirmed with full-scope access (§10 item 7), stack decisions locked (§2).

Hour	Task	Deliverable	Ref
1	Scaffold Next.js App Router repo + `@opennextjs/cloudflare` adapter; write `wrangler.jsonc` (D1 + KV bindings); create the D1 database and KV namespace via `wrangler d1 create` / `wrangler kv namespace create`; port design tokens into `styles/tokens.css`	Repo boots locally and deploys an empty page to a Workers preview URL	§2, §3
2	Build `components/ui/` primitives (Button, Input, Textarea, Card, Badge, FormField) 1:1 from the design system's components	Component library ready for every page	§3
3	`lib/config.ts` (capacity number, brand name/domain placeholders, disclosure string) + `lib/content/landing.ts` content module; build 8 of 16 landing sections	Landing page half-assembled, driven by data not hardcoded copy	§6
4	Remaining 8 landing sections + shared `<Disclosure>` component wired into the footer	Landing page content-complete	§6, §7
5	Programme Terms page — 14 items from `lib/content/terms.ts`; disclosure placed at the top of the Terms page too	Both public pages content-complete	§6
6	Application form UI (`/apply`) — all 20 fields, Zod schema shared client/server, Turnstile widget + honeypot field, disclosure rendered directly above the submit control	Form renders; client-side validation and Turnstile both work	§5, §7
7	D1 schema + `drizzle-kit` migration (§4 table); `/api/apply` route — re-validate server-side → verify Turnstile siteverify → honeypot check → insert → return success; client shows confirmation state	End-to-end submit writes a row to D1	§4, §5
8	`lib/email/client.ts` (`worker-mailer` over `cloudflare:sockets`, Hostinger SMTP secrets in `.dev.vars`); wire both branded templates from `docs/email-templates/` with merge-field interpolation; fire both emails from `/api/apply`	Test submission triggers both emails	§5, §5a, §10 item 8
9	`/admin/applications` list + `[id]` detail page reading D1 directly	Team can review the test submission live	§2, §10 item 5

Hour	Task	Deliverable	Ref
	(status/score/notes editable); gate the route with a custom login page + session cookie (`ADMIN_USERNAME` / `ADMIN_PASSWORD` secrets — code, not dashboard config)		
10	Deploy to a Workers staging URL via GitHub-connected Workers Builds; add `sitemap.ts` / `robots.ts` / per-page `metadata`; run one full submit → email → admin smoke test; spot-check contrast and keyboard nav; write `.dev.vars.example` + a short setup README	Shareable staging link; brief's QA checklist run once, end-to-end	§8, §9, §12

Deferred past hour 10 (called out on purpose, not silently dropped): - Cloudflare Rate Limiting rule / KV counter fallback on `/api/apply` — Turnstile + honeypot cover spam for a demo; add this before any real traffic. - Full WCAG AA contrast verification and a real-device mobile pass — hour 10 only spot-checks these, §9's full pass still needs doing. - Custom domain binding — blocked on the `[Halal Brand]` decision regardless of timeline (§1, stack table). - Cross-client email rendering QA (Outlook/Gmail/Apple Mail) — templates are previewed in Chrome only so far; run them through a real inbox test before relying on them. - The §10 item 8 SMTP spike is folded straight into hour 8 rather than run separately — the highest-risk hour in this schedule. If `worker-mailer` doesn't authenticate cleanly against Hostinger on the first try, expect it to eat into the hour 9–10 buffer.

12. Mapping to the brief's QA Review Criteria

All 8 QA points are addressed directly by the design above: §6 (hero/disclosure ordering, all 16 sections, capacity single-value), §5 (form completeness + consent + confirmation + arrival within a minute), and the brief's "code on GitHub" point — already true, sensibly organised per §3, installable without personal accounts beyond a `.dev.vars.example` and documented `wrangler` setup steps.